Build a Logger with a Decorator

Log every call's arguments, result, duration and errors without touching the function body.

Version 1 – log calls and results

```
import logging, sys
from functools import wraps

logging.basicConfig(
    level=logging.INFO, stream=sys.stdout,
    format="%(levelname)s %(message)s")
log = logging.getLogger("app")

def logged(fn):
    name = fn.__name__
    @wraps(fn)
    def wrapper(*args, **kwargs):
        log.info("call %s args=%s kwargs=%s",
                 name, args, kwargs)
        result = fn(*args, **kwargs)
        log.info("%s returned %r", name, result)
        return result
    return wrapper

@logged
def add(a, b=0):
    return a + b

add(2, b=3)
```

OUTPUT

```
INFO call add args=(2,) kwargs={'b': 3}
INFO add returned 5
```

Version 2 – add timing and errors (times vary)

```
import time

def logged(fn):
    name = fn.__name__
    @wraps(fn)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        try:
            result = fn(*args, **kwargs)
        except Exception as e:
            log.error("%s failed: %r", name, e)
            # keep the traceback
            raise
        ms = (time.perf_counter() - t0) * 1000
        log.info("%s ok in %.1f ms", name, ms)
        return result
    return wrapper

@logged
def divide(a, b):
    return a / b

divide(8, 2)
try:
    divide(1, 0)
except ZeroDivisionError:
    pass
```

OUTPUT

```
INFO divide ok in 0.0 ms
ERROR divide failed: ZeroDivisionError('division by
zero')
```

Design notes

- Use the **`logging`** module, not `print`: levels, handlers, timestamps, files.
- Pass values as `%s` arguments – formatting is skipped if the level is disabled.
- Bare **`raise`** re-raises the original error with its traceback intact.
- Always `@wraps(fn)` so logs show the real function name.

Make the level configurable

```
def logged_at(level):
    def deco(fn):
        @wraps(fn)
        def wrapper(*a, **kw):
            log.log(level, "-> %s%r", fn.__name__, a)
            return fn(*a, **kw)
        return wrapper
    return deco

@logged_at(logging.WARNING)
def risky(x):
    return 10 // x

print(risky(5))
```

OUTPUT

```
WARNING -> risky(5,)
2
```

Log only the slow calls

```
def slow_only(limit_ms):
    def deco(fn):
        name = fn.__name__
        @wraps(fn)
        def wrapper(*a, **kw):
            t0 = time.perf_counter()
            result = fn(*a, **kw)
            ms = (time.perf_counter() - t0) * 1000
            if ms > limit_ms:
                log.warning("%s: %.0fms", name, ms)
            return result
        return wrapper
    return deco

@slow_only(10)
def nap():
    time.sleep(0.03)

nap()
```

OUTPUT

```
WARNING nap: 30ms
```

PITFALL Never log secrets – mask passwords, tokens and card numbers before they reach the log. Remember `async def` functions need an `async` wrapper (`await fn(...)`).

TIP Use `time.perf_counter()` for durations (monotonic, high resolution) – not `time.time()`.